

ONE CHUNK LENGTH DOES NOT FIT AN EPISODE: STABILITY REGIMES FOR ADAPTIVE ACTION CHUNK- ING IN IMITATION LEARNING

Anonymous authors

Paper under double-blind review

ABSTRACT

Action chunking lets imitation policies execute several actions from one inference call. The executed chunk length sets a tradeoff: long chunks are smooth and cheap, short chunks limit how far error compounds before the policy looks again. Practice picks one chunk length per task. We argue this is the wrong unit, and study the quantity it trades against directly. For each state we measure a finite-time maximal Lyapunov exponent by restoring the simulator, perturbing the first action, replaying the rest open-loop, and fitting the growth rate of the divergence. The paper then makes three claims. First, a single task contains both stability regimes: across more than twenty datasets on three platforms, between 78 and 100 percent of episodes contain both stable and unstable segments with several crossings each, the unstable segments are short and concentrated at contact, and the structure is unchanged by a fourfold change in probe size. Second, the regime can be labeled and predicted: small heads reading only camera frames and proprioception reach 0.83 to 0.96 AUROC on confident stamps and transfer across tasks, well above a privileged contact baseline, while a simulator oracle bounds what perfect prediction is worth. Audited against our labels, the replan signals used by existing adaptive-horizon methods do not track measured stability, and a kinematic one is anti-correlated with it. Third, we ask whether the predicted regime can set chunk length at inference time, and report an incomplete and so far negative answer: learned predictors gain 3 to 8 points over a fixed 16-step chunk but stay inside run-to-run noise, never beat a properly swept constant chunk, and a paired test that forks each episode at a measured unstable moment finds that closing the loop rescues as many episodes as it breaks. Chunked execution has a legible and predictable stability structure; converting it into a chunk-length controller does not yet pay, and we identify the closed-loop axis as the measurement that would say when it should.

1 INTRODUCTION

Premise: error compounds along the horizon, and chunk length is how a policy trades against it. Imitation-learned manipulation policies increasingly predict chunks of actions rather than single steps (Zhao et al., 2023; Chi et al., 2023). A chunked executor plays k actions open-loop and then replans. The reason to make k large is that replanning at every step is expensive, adds dithering, and is often unnecessary. The reason to keep k small is the classical one: a policy that acts without observing accumulates error, and in imitation learning that error compounds along the horizon rather than staying bounded (Ross & Bagnell, 2010; Ross et al., 2011). Chunk length is therefore a direct control on how much compounding the executor allows between observations.

How fast error actually compounds is a property of the dynamics at the current state, and it has a standard measurement. If a small perturbation is injected and the system is run forward, the exponential rate at which the perturbed and unperturbed trajectories separate over a finite window is a finite-time maximal Lyapunov exponent (Haller, 2015), a tool already used to diagnose the sensitivity of model predictive control and reinforcement learning policies (Krishna et al., 2023). We write this rate λ and use it as the definition of open-loop stability for a chunk. A segment with λ below a task-specific floor is open-loop stable, meaning a perturbation dies out while the chunk

plays; a segment above it is open-loop unstable, meaning every additional blind step makes the error larger.

Existing methods that vary the execution horizon do not measure this quantity. They trigger on model confidence, on attention weights, on entropy, on the consistency of successive chunk samples, or on the kinematics of the predicted path (Zhao et al., 2026; Wang et al., 2026; Nie et al., 2026; Zhang et al., 2026; Feng et al., 2026; So et al., 2025). Those are proxies for how sure the policy is, not for how fast the plant amplifies an error. This paper studies the quantity itself.

The three parts of this paper. The paper makes three claims and reports the evidence for each separately, including where that evidence is still incomplete. Figure 1 states all three in one picture.

Claim 1: a single task contains both regimes, so a single chunk length per task is the wrong unit. Error does not compound at a uniform rate along an episode. We measure λ at every timestep of more than twenty datasets on three platforms, and find that between 78 and 100 percent of individual episodes contain both open-loop stable and open-loop unstable segments, with several crossings per episode. The unstable segments are short and concentrated at contact events. Chunk length should therefore follow the regime rather than the task. The specific case where this matters is a segment that is open-loop unstable but closed-loop stable: running blind through it compounds error, while replanning every step would not, so $k=1$ is the right answer there and $k=16$ elsewhere.

Claim 2: the regime can be labeled and predicted. We give a labeling algorithm that assigns λ to every state without training anything and without access to policy internals, and we extend it to the closed-loop axis by letting the policy replan inside the perturbed branch. We then show the regime is predictable from what a robot can actually observe. Three predictors are compared: a simulator oracle that measures λ exactly and serves as the upper bound, and two learned heads that read camera frames and proprioception only, one using a single 0.8 second clip and one using 3.2 seconds of history. The learned heads reach 0.83 to 0.96 AUROC on confident stamps and transfer across tasks, well above a privileged contact-flag baseline.

Claim 3: the predicted regime can set chunk length at inference time, and beat a fixed chunk. This is the claim for which our evidence is incomplete, and we report it as such. Learned predictors driving the switch raise mean success over the vanilla policy by 3 to 8 points across four RoboCasa tasks, and they never collapse the way constant single-step execution does. But those gains sit inside the run-to-run spread of this benchmark, no controller beats the best constant chunk length on the one task where a shorter constant genuinely helps, and a paired test that forks each episode at a measured unstable moment finds that closing the loop there rescues as many episodes as it breaks. We report the negative result together with the two deployment failures we diagnosed along the way, both of which are invisible to offline detection metrics.

Contributions.

- A measurable, training-free, policy-agnostic definition of within-episode stability regimes for chunked policies, with an automatic per-task threshold, and evidence across three platforms and more than twenty datasets that both regimes coexist inside single episodes.
- Evidence that the regime is predictable from perception alone, with a simulator oracle that bounds what perfect prediction is worth, and an ablation showing that temporal history does not help, which locates the signal in the current configuration.
- An honest evaluation of regime-conditioned chunk length at inference time, including a paired causal test that isolates the intervention from episode variance, and two calibration failures that offline metrics hide.

2 RELATED WORK

Action chunking policies. Chunked action prediction is standard in modern imitation learning. ACT predicts chunks with a transformer and blends them with temporal ensembling (Zhao et al., 2023). Diffusion Policy denoises an action sequence and executes a prefix before replanning (Chi et al., 2023). Generalist vision-language-action models such as GR00T N1 produce 16-step chunks

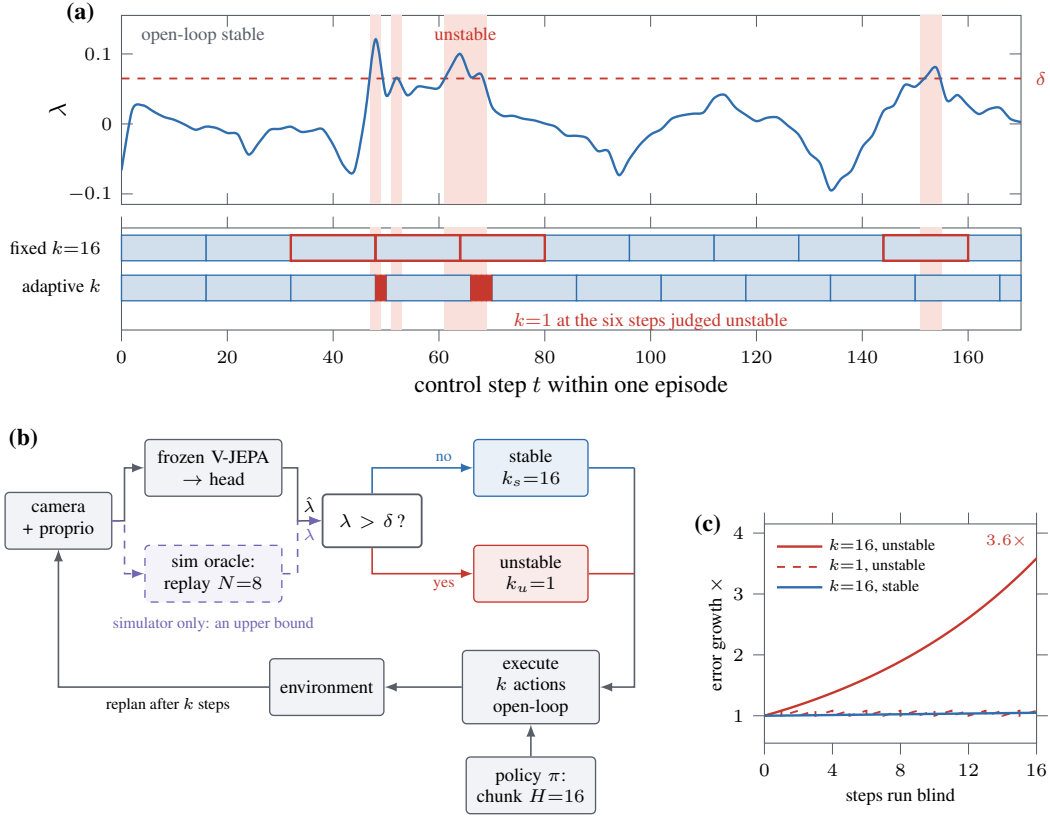


Figure 1: The framework and the three claims. (a) Open-loop divergence rate λ along one real PickPlaceCounterToCabinet episode, with the automatic threshold δ and the unstable segments shaded. Both regimes occur inside this single episode, which contains four separate unstable segments; that is the ordinary case, as 78 to 96 percent of RoboCasa episodes contain both. The ribbons below show what a chunk length can do about it. A fixed $k=16$ schedule is blind to the regime, and four of its eleven chunks (red outline) run straight through an unstable segment. The adaptive schedule is the (16, 1) rule of Section 3 executed on this trace: it keeps the long chunk in stable stretches, where a long chunk costs nothing, and drops to $k=1$ only at the six steps it judges unstable. (b) The controller. At each replan a stability score is obtained either from a learned head reading camera frames and proprioception, which is what a real robot can do, or from a simulator oracle that measures λ by saving the state, replaying the proposed chunk once unperturbed and $N=8$ times perturbed, and rewinding. The score is compared to δ and selects the chunk length; everything else in the loop is unchanged. (c) Why the regime has to be known. Growth of the perturbation over a chunk at the measured median rates for this task, $\lambda=.080$ on unstable stamps and $.003$ on stable ones. Running blind through an unstable segment compounds error by $3.6\times$ over 16 steps, while the same 16 blind steps in a stable segment cost $1.05\times$; replanning every step holds the unstable case at $1.08\times$. Over the full $K=24$ label window the measured divergence ratio is $6.6\times$ to $14\times$ on unstable stamps against $0.97\times$ to $1.4\times$ on stable ones across the four tasks. A single chunk length must pay the first cost everywhere or give up the second everywhere.

from a flow-matching head (NVIDIA, 2025). In all of these the executed chunk length is a fixed hyperparameter, chosen once per task.

Compounding error and the theory of chunking. The reason short chunks help at all is the compounding of error under a policy that acts without observing, analyzed for imitation learning by Ross & Bagnell (2010) and Ross et al. (2011), whose bound grows quadratically in the horizon in the worst case. In continuous action spaces the situation is sharper: Simchowitz et al. (2025) show that even with exponentially stable dynamics and a smooth deterministic expert, any smooth imitator can suffer execution error exponential in the horizon relative to its error on the training distribution.

Closest to this paper, Zhang et al. (2025) prove that action chunking is a remedy, because a chunk longer than a threshold set by the contraction rate restores exponential stability of the learned closed loop and makes the trajectory error horizon-free. Their analysis and ours meet at the same inequality and pull in opposite directions on the same knob. Theirs is a lower bound on chunk length, derived under a single contraction rate for the system; ours is the upper bound that the same inequality implies when the local rate is positive, and our measurement is the state-dependent counterpart of their constant. Section 3 makes the connection explicit, and Claim 1 is the empirical observation that no single rate describes an episode.

Adaptive execution horizons. A recent line of work varies the executed horizon at test time, and the distinguishing feature is the signal each one switches on. Bidirectional decoding re-samples chunks and selects for closed-loop coherence (Liu et al., 2024). DEHP trains an execution-horizon branch with chunk-level reinforcement learning over a frozen base policy (Zhao et al., 2026). AutoHorizon reads the policy’s action self-attention as a confidence signal (Wang et al., 2026). PACE cuts chunks at low-speed valleys of the predicted trajectory (Nie et al., 2026). DVAC uses the variance of the clean-action estimate along the denoising trajectory of a flow policy (Feng et al., 2026), and SGAC compares the queued chunk against a freshly sampled one (So et al., 2025). HiPolicy predicts chunks at several frequencies and gates them by entropy (Zhang et al., 2026), and Wen et al. (2026) learn chunk length end-to-end from visual context.

These methods share our motivation that manipulation is stage-structured, and PACE states that intuition explicitly. The difference is what gets measured. Confidence, attention, entropy and sample agreement describe the policy’s uncertainty about its own output. Speed valleys describe the kinematics of the predicted path. None of them measure how fast the plant amplifies an action error, which is the quantity chunk length actually trades against, and none produce per-timestep labels that exist independently of the controller that consumes them. We measure these proxies against λ at the same states in Section 6.3.

Stability analysis in control and learning. Finite-time Lyapunov exponents are a classical tool for diagnosing trajectory divergence and coherent structure in flows (Haller, 2015), and have been used to analyze the sensitivity of MPC and reinforcement learning policies (Krishna et al., 2023). We bring the tool to a different question: not whether a controller is stable overall, but which chunk length a chunked policy should execute, segment by segment. To our knowledge no prior work labels states with measured open-loop divergence, separates open-loop from closed-loop stability per segment, or uses such labels to schedule the execution horizon.

3 MEASURING AND PREDICTING STABILITY REGIMES

Setup. A policy π maps an observation o_t to an action chunk $(a_t, \dots, a_{t+H-1})$ of nominal length H , here $H=16$. The executor plays the first $k \leq H$ actions open-loop and then replans. We ask how k should depend on the state.

Premise: what an open-loop chunk costs. The price of executing k actions blind can be written down before it is measured. Take the standard incremental-stability model of the plant: the dynamics are (C, ρ) -exponentially input-to-state stable if any two trajectories driven by input sequences u and u' satisfy

$$\|x_j - x'_j\| \leq C \rho^{j-1} \|x_1 - x'_1\| + C \sum_{i=1}^{j-1} \rho^{j-1-i} \|u_i - u'_i\|, \quad (1)$$

with contraction rate ρ and constant $C \geq 1$ (Simchowitz et al., 2025; Zhang et al., 2025). The two things a chunked executor does are the two special cases of equation 1.

Perturb one input. Start both branches at the same state and perturb only u_1 , by σ_u . Then equation 1 collapses to a single term, $\log \|x_j - x'_j\| \leq \log C \sigma_u + (j-1) \log \rho$, a straight line in j of slope $\log \rho$. The labeling procedure below perturbs exactly the first action and fits exactly that slope, so the λ it returns is a state-local empirical estimate of $\log \rho$ in units of inverse control steps, and the linear fit is the model rather than a convenience.

Execute a chunk. A policy whose per-step action error is ε commits that error at every step of the chunk, since nothing corrects it until the next replan. Putting $\|u_i - u'_i\| = \varepsilon$ in equation 1 and writing $\rho = e^\lambda$, the deviation at the end of a chunk of length k obeys

$$\|x_k - x'_k\| \leq C\varepsilon \frac{e^{\lambda k} - 1}{e^\lambda - 1} = \begin{cases} C\varepsilon / (1 - e^\lambda) & \lambda < 0, \\ C\varepsilon k & \lambda = 0, \\ C\varepsilon e^{\lambda k} / (e^\lambda - 1) & \lambda > 0. \end{cases} \quad (2)$$

Chunk length is free where the segment is open-loop stable, costs linearly at the noise floor, and costs exponentially above it. The scale is not academic: the two states in Figure 2(b) carry $\lambda=.006$ and $\lambda=.121$, so a single 16-step chunk multiplies a given action error by 1.1 at the first and by 6.9 at the second.

Over an episode. Run T/k chunks and let a replan contract the accumulated deviation by $\gamma < 1$ where the policy is closed-loop stable. The deviation at chunk boundaries then follows a linear recursion of gain $\gamma e^{\lambda k}$, which stays bounded over the episode exactly when $k < \log(1/\gamma)/\lambda$. Open-loop instability sets a ceiling on chunk length, and closed-loop stability is what raises that ceiling.

Two known results bracket this picture. With no assumption on the dynamics at all, the classical imitation bound is $J(\hat{\pi}) \leq J(\pi^*) + T^2\varepsilon$ for a policy with per-step error ε , and the quadratic is tight (Ross & Bagnell, 2010). With continuous actions it is worse: Simchowit et al. (2025) show that even under equation 1 with $\rho < 1$ and a smooth deterministic expert, any smooth imitator can suffer execution error $\exp(\Omega(T))$ times its error on the training distribution. In the other direction Zhang et al. (2025) prove that chunking is the remedy: once the chunk is longer than $\log(1/\rho)^{-1} \log(\text{poly}(L, C))$ the chunked closed loop is exponentially stable again, with rate $\rho^{1/2}$, and its trajectory error becomes horizon-free. That condition is a *lower* bound on chunk length, and it assumes a single $\rho < 1$ for the system.

The bound in equation 2 is the other side of the same inequality, and it is why this paper measures rather than assumes. An admissible chunk length has to satisfy both bounds at once,

$$\underbrace{\frac{\log \text{poly}(L, C)}{\log(1/\rho)}}_{\text{fixed by the system}} < k < \underbrace{\frac{\log(1/\gamma)}{\lambda(s_t)}}_{\text{moves with the state}}, \quad (3)$$

and Claim 1 is the finding that the right-hand side is not a constant: λ takes both signs inside a single episode, so a single k per task must respect the tightest ceiling the episode ever imposes, leaving it conservative wherever λ is small and unsafe wherever λ is large. The switching rule of this paper is the two-level approximation of equation 3. Neither equation 2 nor equation 3 is a theorem from the works cited; both are the local reading of equation 1 that a per-state measurement of λ makes available, and γ in particular is a modelling assumption about what one replan buys, which we do not measure. Appendix A gives the derivation, the statements quoted from, and what the correspondence between λ and $\log \rho$ does and does not license.

Open-loop stability labels. For a state s_{t_0} with actions $a_{t_0}, \dots, a_{t_0+K-1}$ we measure divergence under action perturbation. First, restore the simulator to s_{t_0} and replay the K actions unperturbed, recording a task metric y_j , here end-effector position and gripper joint positions. Second, in N branches, perturb the translation components of a_{t_0} with noise of scale σ_u , replay the same remaining actions, and record $y_j^{(n)}$. Third, fit by least squares the slope λ of $\frac{1}{N} \sum_n \log \|y_j^{(n)} - y_j\|$ against j . This λ is a finite-time maximal Lyapunov exponent for the chunk. We use $K=24$, $N=8$, a stride of 2, and $\sigma_u=0.05$ on RoboCasa. The label is open-loop by construction: no replanning happens inside the window, so λ measures the plant and the task geometry under the recorded behavior, not the reactivity of any controller. The procedure trains nothing and reads no policy internals. Figure 2 shows the construction, the fit on two real states, and the distribution the threshold below is read from. Appendix B gives the procedure as pseudocode, the per-platform task metric, and the reproducibility details, and Appendix C covers how σ_u is set and how much the labels depend on it.

Automatic threshold. Divergence slopes have a task-specific noise floor from contact chatter and metric scale. We set $\delta = P_{95}(|\lambda| : \lambda < \text{median}(\lambda))$, the 95th percentile of the magnitude of the below-median half, and call a stamp unstable if $\lambda > \delta$. This adapts to each task’s floor with no

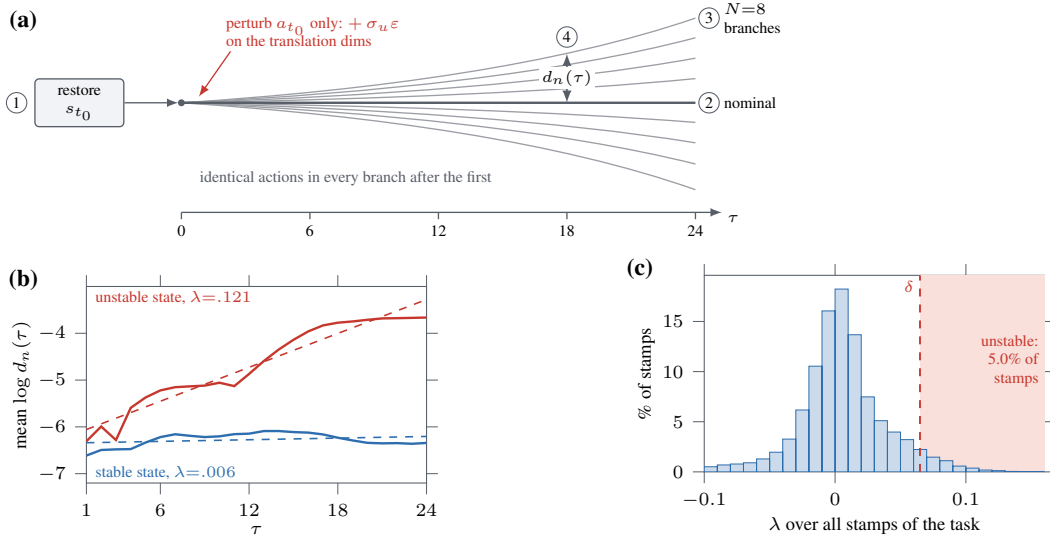


Figure 2: **How one stability label is produced.** (a) The procedure at a single state, and the whole of it: restore the simulator to s_{t_0} (1); replay the $K=24$ recorded actions unperturbed to get the nominal branch (2); repeat $N=8$ times with Gaussian noise added to the translation components of the first action only, every branch replaying the same recorded actions thereafter (3); and record the task-space distance $d_n(\tau)$ between each branch and the nominal at every step (4). The label λ is the least-squares slope of the mean log distance against τ . Nothing is trained, no policy internals are read, and the policy is not involved at all: the label describes the plant and the task geometry under the recorded behavior. (b) The fit, on two real states drawn from the same episode as Figure 1, with the least-squares line dashed. At the unstable state the branches separate from 1.8 to 25.6 millimetres in the task metric over the window, a factor of 14; at the stable state they stay between 1.3 and 1.8 millimetres, and the fitted slope is twenty times smaller. The label is the slope, not the final distance, so it does not reward states that merely start far apart. Neither curve is a clean straight line, and the fit is a summary of the window rather than a claim that growth is exactly exponential within it. (c) Where the threshold comes from. Distribution of λ over all 23,724 stamps of PickPlaceCounterToCabinet. The bulk sits at a task-specific noise floor set by contact chatter and metric scale, so a fixed cutoff would not transfer between tasks; δ is instead read off the below-median half of this distribution as $P_{95}(|\lambda|)$, which puts 5.0% of stamps in the unstable class here. Stamps with $|\lambda| \leq \delta$ are the deadband and are excluded from classification training.

manual tuning. Stamps with $|\lambda| \leq \delta$ form a deadband where the label is ambiguous by construction, and we exclude them from classification training and report classification quality both with and without them.

Closed-loop stability labels. The same construction gives a second axis if the perturbed branch is driven by the policy replanning at every step instead of replaying fixed actions. Two choices matter. The nominal branch must also be policy-driven and unperturbed, otherwise the measured rate mixes perturbation growth with the ordinary difference between the policy and the recorded behavior. And because modern policies sample stochastically, two unperturbed policy rollouts from one state already separate, so we run a control pair from the same state with no perturbation and report its rate λ_{ctrl} alongside. A segment is closed-loop stable when the perturbation adds nothing beyond that floor.

Two axes, four regimes. Open-loop stability says whether errors grow while a chunk plays out. Closed-loop stability says whether the policy tolerates frequent replanning, since some tasks degrade sharply at $k=1$ when replanning every step introduces dithering near fine goals. The two axes give four regimes. Open-loop stable and closed-loop stable: any k works, prefer long chunks. Open-loop unstable and closed-loop stable: the case that motivates switching, because short chunks fix what long chunks break. Open-loop stable and closed-loop unstable: long chunks are mandatory and a

switcher must not produce false positives here. Unstable on both axes: no chunk length helps and the failure is not a chunking problem.

Predictors. To act on the signal at rollout time without a simulator we train a small attentive head on frozen V-JEPA features that regresses λ and classifies $\lambda > \delta$. The h1 variant reads one clip of 16 consecutive frames, about 0.8 seconds, together with the robot state over the same window. The h4 variant reads four such clips spaced 16 frames apart, about 3.2 seconds, with a learned per-clip position embedding, and tests whether the regime is visible in motion over time. We also evaluate a simulator oracle that runs the labeling procedure live at each replan and therefore measures λ instead of predicting it. The oracle needs to roll the simulator forward and rewind it, so it cannot run on a real robot; its role is to bound what perfect prediction would be worth. Appendix E gives the architecture and training setup.

Switching rule. At each replan the controller obtains a stability score and executes $k = k_u$ if the score exceeds a threshold, else $k = k_s$, with $(k_s, k_u) = (16, 1)$ throughout unless stated. The rule only shortens on predicted instability, so in stable segments it costs nothing by construction. Setting the threshold for a learned head is not trivial and we treat it as part of the method rather than an implementation detail; Section 7 reports the two ways it fails and the calibration we adopt.

4 EXPERIMENTAL DESIGN

Platforms and tasks. Three platforms with different policy classes and scales. (1) RoboCasa kitchen tasks (Nasiriany et al., 2024) with the public GR00T N1.5 3B multitask checkpoint, nothing trained by us: OpenCabinet, OpenDrawer, PickPlaceCounterToCabinet and TurnOnSinkFaucet, with 102,348 labeled stamps from 800 human demonstrations. We also train ACT policies (Zhao et al., 2023) on the same demonstrations as a second policy class. (2) LIBERO-Long (Liu et al., 2023) with per-task diffusion policies (Chi et al., 2023). (3) MimicGen (Mandlekar et al., 2023) and robomimic tasks with diffusion policies. Every evaluation cell is 50 episodes, and every switching evaluation includes the fully closed-loop branch ($k_s=16, k_u=1$).

Noise floor. Success rates on 50 episodes carry a sampling standard error near 7 points, so the difference between two independent 50-episode runs has a standard error near 10 points. We measure this directly rather than assuming it. Pairs of runs that execute identical behavior, a fixed-16 run and a shadow run that scores the signal without acting on it, differ by 6, 8, 18 and 4 points on the four RoboCasa tasks, and two seeds of one identical switching configuration differ by 6 points. We therefore treat differences below about 10 points on a single pair of cells as unresolved, and say so wherever it applies.

Experiments per claim. For Claim 1, offline labeling of all datasets, a probe-size sweep at $\sigma_u \in \{0.025, 0.05, 0.10\}$, and online shadow runs that score every replan of a fixed-16 policy without acting, which give the regime distribution on the states the policy actually visits. For Claim 2, predictor training on held-out demonstrations with three seeds per configuration, a privileged contact-flag baseline, cross-task transfer, and a comparison of competing replan signals measured at the same states. For Claim 3, fixed- k sweeps, oracle switching, learned switching with h1 and h4, a rate-matched random control that fires at the same rate but at random replans, and a paired fork test that removes episode variance by running both decisions from one simulator state.

5 CLAIM 1: ONE TASK CONTAINS BOTH REGIMES

5.1 BOTH REGIMES COEXIST INSIDE SINGLE EPISODES

Table 1 summarizes the RoboCasa labels. Unstable stamps are a small minority everywhere, between 2 and 5 percent of all timesteps, but they are not spread evenly across episodes. Between 78 and 96 percent of individual episodes contain both regimes, and a typical episode crosses between them several times. The same procedure on LIBERO, MimicGen and robomimic gives the same picture on twenty more datasets, with unstable fractions from 4 to 51 percent, both regimes present

Task	stamps	δ	unstable	episodes with both	crossings
OpenCabinet	34,133	.085	2.3%	86%	4
OpenDrawer	24,293	.076	3.1%	78%	2
PickPlaceCounterToCabinet	23,724	.065	5.0%	96%	6
TurnOnSinkFaucet	20,198	.058	3.9%	82%	2

Table 1: Open-loop stability labels on RoboCasa, 200 demonstrations per task. δ is the automatic per-task threshold and unstable is the fraction of stamps with $\lambda > \delta$. The last two columns show the two regimes coexisting inside single episodes: the fraction of episodes containing both, and the median number of crossings between them per episode.

Dataset	stamps	δ	regime share, all stamps		heterogeneity within episodes	
			deadband	unstable	within-ep var	crossings/100
<i>RoboCasa, GR00T N1.5</i>						
OpenCabinet	34,133	0.085	95%	2.3%	99.4%	2.3
OpenDrawer	24,293	0.076	94%	3.1%	97.8%	2.4
PickPlaceToCabinet	23,724	0.065	93%	5.0%	97.8%	5.2
TurnOnSinkFaucet	20,198	0.058	94%	3.9%	94.5%	2.8
<i>LIBERO-10, diffusion policy</i>						
K3 stove + moka	6,060	0.085	93%	4.4%	96.9%	4.1
K4 bowl in drawer	5,632	0.024	68%	29.8%	98.0%	9.9
K6 mug in microwave	7,031	0.015	47%	50.9%	96.6%	22.8
K8 both moka pots	9,812	0.014	47%	50.1%	98.1%	21.5
L1 soup + cream	6,147	0.025	73%	25.0%	97.6%	12.2
L2 soup + tomato	6,761	0.014	48%	49.8%	97.0%	21.3
L2 cream + butter	5,920	0.014	48%	50.3%	97.0%	25.1
L5 mug + chocolate	5,868	0.018	54%	43.1%	98.5%	11.5
L6 mug + chocolate	5,789	0.018	60%	37.7%	96.5%	14.6
S1 book on shelf	4,146	0.011	47%	50.2%	91.9%	15.8
<i>MimicGen, diffusion policy</i>						
coffee preparation	66,724	0.064	85%	12.2%	99.6%	9.2
nut assembly	33,251	0.056	83%	14.8%	98.8%	12.8
square	12,874	0.036	61%	36.3%	95.5%	17.5
stack	8,445	0.022	55%	42.7%	95.7%	23.2
<i>robomimic, diffusion policy</i>						
can	9,254	0.050	83%	14.2%	95.9%	12.8
lift	2,485	0.018	55%	42.1%	95.5%	23.2
square	12,723	0.037	64%	33.4%	94.8%	15.2
tool_hang	45,633	0.035	57%	40.3%	98.2%	19.1
<i>policy rollouts (not demonstrations)</i>						
can (rollouts)	8,360	0.038	66%	31.6%	89.7%	17.5
lift (rollouts)	2,585	0.032	65%	32.7%	82.3%	14.3
closed-loop axis: measured on the four RoboCasa tasks (Table 7); [pending; not yet measured on the other 20]						

closed-loop axis: measured on the four RoboCasa tasks (Table 7); [pending: not yet measured on the other 20]

Table 2: Every dataset we labeled, 24 in total. δ is the automatic per-task threshold, unstable is the share of stamps with $\lambda > \delta$, and deadband is $|\lambda| \leq \delta$ where the label is ambiguous by construction. We omit the stable share because the threshold rule fixes it near 2.5% on every dataset, so it carries no information. The last two columns measure heterogeneity. Within-ep var is the share of the total variance of λ that survives removing the between-episode means, so a value near 100% means stability varies inside episodes rather than between them; it is 82 to 99.6% on every dataset, which is the central evidence that chunk length should not be a per-task constant. Crossings/100 counts regime transitions per 100 labeled stamps, normalised so longer episodes do not inflate it: the kitchen tasks sit at 2 to 5 and alternate rarely, while long-horizon and high-precision tasks sit at 10 to 25.

in essentially every episode, and 3 to 43 crossings per episode. Error therefore does not compound at a uniform rate along an episode, which is what makes one chunk length per task the wrong unit.

The unstable segments are short and localized. Measured on the states the policy itself visits, a contiguous unstable stretch lasts a median of 1 to 5 control steps and occupies 1 to 3 percent of an episode on three of the four tasks. OpenDrawer is the exception, where the signal stays up for long stretches covering about 18 percent of an episode, and we return to that task as the case where switching degenerates.

<i>RoboCasa PickPlaceToCabinet</i> , $\delta=.065$	
demo 52, 6 crossings	UUU
demo 152, 26 crossings	U
<i>RoboCasa TurnOnSinkFaucet</i> , $\delta=.058$	
demo 131, 2 crossings	U
demo 68, 26 crossings	..U..U..U..UUUU.....U.....UU
<i>LIBERO K8, both moka pots</i> , $\delta=.014$	
demo 15, 43 crossings	UU..UUUUU.....UUUUUUUUUU..UUUUUUUUUU
demo 33, 73 crossings	..UU.....UU..UUUUUU.....U..U..U..U..UU..U..UUUU.....U.....UU
<i>robomimic tool_hang</i> , $\delta=.035$	
demo 42, 43 crossings	UU.....U..UU..UUU.....UU..U..UUU..UUU..UUUU
demo 156, 79 crossings	UUUUUU..U..U..U..UU..U..UU.....UUUUUU..UUUU..U..U

Figure 3: Regime timelines for individual demonstrations, each row one episode resampled to 56 columns. A dot is a stable or deadband stamp and the letter U is an unstable one. The kitchen tasks put one or two short unstable islands in a long stable episode. The long-horizon LIBERO task and the high-precision tool-hang task alternate throughout, which is the regime where a shorter constant chunk, not switching, is the right answer.

Task	$k=16$	$k=8$	$k=4$	$k=1$
OpenCabinet	.52	.52	.44	.48
OpenDrawer	.64	.56	.64	.40
PickPlaceCounterToCabinet	.58	.78	.70	.52
TurnOnSinkFaucet	.42	.36	.10	.04

Table 3: RoboCasa with GR00T N1.5, success over 50 episodes per cell. Bold marks the best constant chunk length per task.

5.2 THE STRUCTURE IS A PROPERTY OF THE DYNAMICS, NOT OF THE PROBE

A reader may ask whether the regimes are an artifact of the perturbation size we inject. They are not. Relabeling all four tasks at σ_u of 0.025 and 0.100, a fourfold range around the 0.05 used throughout, leaves the per-stamp ordering of λ almost unchanged, with Spearman correlation 0.95 to 0.98 between adjacent settings and 0.82 to 0.90 between the extremes, and the unstable flag agrees on 97 to 99 percent of stamps.

5.3 FIXED CHUNK LENGTH FOLLOWS THE REGIME PROFILE

Table 3 gives the fixed- k sweep. The best constant is 16, 16, 8 and 16 across the four tasks, so no single value is right even inside one benchmark. TurnOnSinkFaucet is the extreme case, collapsing from .42 to .04 as k shrinks, which is the signature of a task that cannot tolerate constant replanning. A second policy class shows the same task dependence with a different profile. ACT policies trained on the same demonstrations score .58/.64/.52/.32 on OpenCabinet, .30/.32/.04/.10 on OpenDrawer, .12/.12/.04/.10 on PickPlaceCounterToCabinet and .50/.40/.34/.36 on TurnOnSinkFaucet for k of 16, 8, 4 and 1, so ACT tolerates single-step execution on the faucet task where the generalist policy does not.

6 CLAIM 2: THE REGIME CAN BE LABELED AND PREDICTED

6.1 THREE WAYS TO ANSWER THE SAME QUESTION

Every controller in this paper answers one yes or no question at each replan: is the chunk the policy just proposed open-loop unstable? If the answer is no, the controller executes all 16 actions without looking. If the answer is yes, it switches to single-step execution and replans after every action until the signal clears. The controllers differ only in how they reach that answer.

The **simulator oracle** measures the answer instead of predicting it. At each replan it saves the simulator state, replays the proposed chunk once unperturbed and eight more times with the first action perturbed, fits λ , restores the state exactly, and compares λ to δ . It is trained on nothing. It

		AUROC, confident stamps		recall at matched budget	
Platform	Task	h1	h4	h1	h4
RoboCasa GR00T	OpenCabinet	0.885 ± 0.021	0.858 ± 0.014	0.33	0.17
RoboCasa GR00T	OpenDrawer	0.863 ± 0.023	0.838 ± 0.010	0.18	0.10
RoboCasa GR00T	PickPlaceToCabinet	0.895 ± 0.016	0.887 ± 0.006	0.25	0.14
RoboCasa GR00T	TurnOnSinkFaucet	0.902 ± 0.022	0.895 ± 0.012	0.20	0.15
MimicGen DP	coffee preparation	0.939 ± 0.006	0.953 ± 0.002	–	–
MimicGen DP	nut assembly	0.885 ± 0.029	0.859 ± 0.022	–	–
<i>other evidence, single-clip architecture</i>					
LIBERO-10 DP	10 tasks	.843 to .913		–	
robomimic DP	4-task pool	.868		–	
robomimic DP	square $\rightarrow$ tool.hang	.719 to .768		–	
robomimic DP	DINOv2, single frame	.859		–	
any	privileged contact flag	.602 to .802		–	
h1 and h4 on the remaining 16 labeled datasets: [pending: in the training pipeline]					
pooled head over all four RoboCasa tasks: [pending: in the training pipeline]					
head trained on both stability axes jointly: [pending: not started, waiting on closed-loop labels]					

Table 4: Detection quality of the stability predictors. AUROC is measured on held-out demonstrations, mean and standard deviation over three seeds, and restricted to confident stamps whose label is not inside the deadband. Recall at matched budget is the fraction of truly unstable stamps recovered when the firing rate is fixed to the label base rate, where random firing would recover 2.5 to 5.0 percent, so the heads run 3 to 13 times better than chance. Bold marks the better of h1 and h4 per row: extra temporal history helps on one dataset out of six and hurts on the other five. The privileged contact flag, which reports whether the gripper touches a movable object, is the baseline a learned head must beat to show it is doing more than contact detection.

cannot run on a real robot because it rolls the simulator forward and then rewinds it, so its role is to bound what a perfect answer would be worth.

The **h1 predictor** is a small attentive head on frozen V-JEPA features that predicts λ from one clip of 16 consecutive camera frames, about 0.8 seconds, plus the robot state over the same window. It is trained by supervised regression and classification against the labels of Table 1, sees no simulator at run time, and costs one forward pass per replan.

The **h4 predictor** has identical architecture and identical training targets but reads four clips spaced 16 frames apart, about 3.2 seconds of history, with a learned per-clip position embedding.

6.2 PERCEPTION RECOVERS THE REGIME

Table 4 gives detection quality on held-out demonstrations. Both heads rank unstable segments well above chance on all four tasks. The result holds on the other platforms as well: 0.94 to 0.96 AUROC on confident stamps for MimicGen coffee preparation, 0.83 to 0.94 for nut assembly, 0.88 to 0.91 across LIBERO tasks, and 0.87 on a pooled four-task robomimic set, with cross-task transfer from square to tool hang at 0.72 to 0.77.

The heads are not merely detecting contact. A privileged flag that reports whether the gripper touches a movable object reaches only 0.60 to 0.80 AUROC on the same stamps, so the learned heads discriminate which contact moments are sensitive rather than that contact is occurring.

Whether temporal history helps depends on the task, and the dependence is systematic. On the four RoboCasa tasks the four-clip head is worse than the single-clip head everywhere, which suggested at first that history is simply unhelpful. Extending the comparison to 14 datasets reverses that reading: the single-clip head wins on 6 and the four-clip head on 8, with mean AUROC 0.848 against 0.858. What separates them is how fast the task alternates between regimes. Ranking datasets by crossings per 100 stamps from Table 2, the advantage of history correlates with the alternation rate at Spearman +0.64: on the seven slowest-alternating datasets the four-clip head loses by 0.013 on average and wins 2 of 7, while on the seven fastest it gains 0.034 and wins 6 of 7.

Signal family	Spearman with λ	AUROC for $\lambda > \delta$
sample spread (confidence)	-.07 to +.10	.61 to .76
chunk agreement (consistency)	-.17 to .00	.40 to .62
predicted speed valley (kinematic)	+.15 to +.21, inverted	.31 to .69

Table 5: Competing replan signals measured against λ at the same states, ranges across the four RoboCasa tasks. The kinematic row is inverted: commanded speed correlates positively with λ , so firing at speed valleys fires at the more stable moments.

We read this as follows. A single frame suffices when instability arrives as isolated islands, because the regime is then a property of the current configuration, whether the gripper is engaged and how the geometry sits. A single-frame DINOv2 head reaching 0.86 AUROC on confident stamps and proprioception-only heads matching vision-only heads on MimicGen both support that. When regimes alternate several times per hundred timesteps the current configuration is ambiguous and recent motion carries the extra information. RoboCasa, our main platform, sits firmly in the first case at 2 to 5 crossings per 100 stamps, which is why the four-clip head loses there. [pending: 8 of 22 datasets still training; the correlation is over the 14 with both heads complete]

6.3 EXISTING REPLAN SIGNALS DO NOT TRACK THE MEASURED ONE

Because our labels exist independently of any controller, they can be used to audit the signals other methods switch on. We instrument shadow rollouts to record, at the same state and the same replan, our λ alongside the sample spread of repeated chunk draws, the agreement between the queued chunk and a freshly sampled one, and the speed profile of the predicted chunk, which are the signals used by the confidence, consistency and kinematic families respectively. Table 5 reports the result over 1,370 to 2,339 replans per task.

None of the three tracks λ closely. The confidence family carries a weak tail signal but almost no graded correlation. The consistency family is near chance. The kinematic family is the striking case: commanded speed correlates positively with λ , so a rule that fires at speed valleys fires preferentially at the more stable moments, the opposite of what the stability measurement recommends. This does not say those methods do not work, since a trigger can help for reasons unrelated to stability. It says they are measuring something else, which is why our labels are not redundant with them.

Running them as controllers. Correlation says what a signal measures, not what it is worth. We therefore reimplemented each signal as a controller in our own executor, switching between $k=16$ and $k=1$ on the same policy, tasks and episode budget as our own runs, with each threshold set so the signal fires at the oracle’s rate for that task. DEHP is excluded because its trigger is a horizon head trained with reinforcement learning rather than a computable statistic. Table 6 gives the result.

Read with that caveat, the ranking is unremarkable: every learned or heuristic controller lands within about six points of the others, and only the simulator oracle separates from the pack. Nothing here supports a claim that our signal makes a better trigger than the published alternatives at equal budget; what it does support is that none of them, including ours, converts into a large success gain on these tasks.

6.4 BOTH STABILITY AXES MEASURED AT THE SAME STATES

Open-loop stability is one axis. The regime that would justify switching is open-loop unstable together with closed-loop stable, and deciding membership needs the closed-loop measurement of Section 6. We ran it online on all four RoboCasa tasks, at every replan of a fixed-16 rollout, so both axes are measured at the same states on the distribution the policy actually visits. Table 7 is the result over 2,925 paired probes.

Three readings matter. The switching regime exists but is a minority: 7.0 percent of all stamps, which is 22 percent of the open-loop-unstable stamps. Its size tracks each task’s own instability, running 11, 16, 23 and 31 percent of the unstable stamps on OpenCabinet, OpenDrawer, PickPlace-CounterToCabinet and TurnOnSinkFaucet. Closed-loop instability is rare everywhere, 1.0 percent

Controller	signal family	mean	achieved firing	target
simulator oracle	measured λ	.637	.06 to .15	matched
ours, h4	learned λ	.620	.14 to .28	matched
SGAC chunk cosine	consistency	.615	.02 to .08	under
DVAC sample variance	confidence	.585	.06 to .15	matched
BID chunk L2	consistency	.585	.00 to .09	under
ours, h1	learned λ	.570	.15 to .38	over
PACE speed valley	kinematic	.555	.07 to .14	matched
vanilla, fixed $k=16$	none	.540	0	

Table 6: Replan signals run as controllers on GR00T and RoboCasa, mean success over the four tasks at 50 episodes each. Every controller uses the identical executor and differs only in the statistic it thresholds. Achieved firing is the range of realised firing rates across the four tasks. Thresholds were set from a shadow pass in which the signal is scored but never acted on; once a signal acts it changes the states the policy visits, so realised rates drift from their targets. The two consistency signals drifted down, and one cell, BID on OpenCabinet, fired on zero replans and is therefore a fixed-16 run in disguise. Their means are consequently obtained with roughly half our intervention budget, so those two rows are not yet like-for-like and we report the realised rates rather than hide the discrepancy. [pending: recalibrate the consistency signals against their live firing rates and rerun]

open-loop	closed-loop			row
	stable	deadband	unstable	
stable	1.7%	8.7%	0.2%	10.6%
deadband	7.8%	47.4%	2.1%	57.3%
unstable	7.0%	24.1%	1.0%	32.1%
column	16.4%	80.2%	3.3%	

Table 7: Both stability axes at the same states, pooled over the four RoboCasa tasks, 2,925 paired probes from 95 episodes. The closed-loop value is the growth rate of the perturbation minus the rate at which two *unperturbed* policy rollouts separate, so it measures what the perturbation adds beyond the policy’s own sampling. Both axes use the same per-task threshold, so they ask the same question on the same scale. The bold cell is the regime that motivates switching.

pooled and at most 5.6 percent on any task, so on these tasks the four-regime picture is closer to a two-regime one.

The control branch was necessary rather than decorative. On every task, at about 66 percent of stamps the perturbed closed-loop branch grows *more slowly* than two unperturbed rollouts do, meaning the policy’s own sampling separates trajectories faster than our injected perturbation. Without subtracting that floor the closed-loop axis measures diffusion sampling rather than stability, which is the defect in an earlier closed-loop labeling we discarded.

This also sets a ceiling on what any open-loop detector can do. Our predictors are trained on the open-loop axis alone, so even perfect open-loop detection spends about 78 percent of its interventions at moments where closing the loop cannot help. That is a quantitative reason to expect the intervention results of Section 7 to be weak, without the stability signal itself being wrong. Training a head on the conjunction of both axes is the direct consequence and we leave it to future work.

7 CLAIM 3: SETTING CHUNK LENGTH FROM THE PREDICTED REGIME

This is the claim for which our evidence is incomplete, and we report it as such.

[pending: SAMPLE SIZE, to fix before submission. Every cell in this section is 50 episodes at a single seed, which gives a standard error near 7 points on a success rate and near 10 points on the difference between two cells. Published adaptive-chunking work evaluates far more heavily: PACE reports 900 episodes per task-method pair over 50 tasks, DEHP reports 1000 environments over 3 seeds, SGAC averages 100 episodes over 3 seeds. Our present numbers therefore cannot resolve

Task	No stability signal			Measured	Learned, no simulator			
	vanilla $k=16$	always $k=1$	best fixed (k)	oracle (16, 1)	h1 (16, 1)	fired	h4 (16, 1)	fired
OpenCabinet	.52	.48	.52 (16)	.60	.54	.16	.60	.28
OpenDrawer	.64	.40	.64 (16)	.62	.60	.34	.62	.26
PickPlaceCounterToCabinet	.58	.52	.78 (8)	.77	.60	.38	.66	.14
TurnOnSinkFaucet	.42	.04	.42 (16)	.56	.54	.15	.60	.14
mean	.54	.36	.59	.64	.57		.62	

Table 8: RoboCasa with GR00T N1.5, success over 50 episodes per cell. The pair (16, 1) under a switching controller gives its two chunk lengths: it executes the long chunk $k_s=16$ when the segment is judged open-loop stable and drops to the short chunk $k_u=1$ when it is judged unstable. The oracle column is the mean over available seeds, two for PickPlaceCounterToCabinet and TurnOnSinkFaucet and one otherwise. The fired columns give the fraction of replans at which the predictor chose the short chunk. The always $k=1$ column is the opposite extreme from the vanilla policy. The best fixed column names the constant chunk length that achieves it in parentheses, from Table 3; OpenCabinet ties at $k=8$ and OpenDrawer at $k=4$ and we report the longer chunk. Repeated runs of an identical configuration differ by about .10 at this sample size.

effects of the size those papers report, and the final version must raise every cell in Tables 8 and 9 to at least 3 seeds of 50 episodes, and preferably widen the task set, before any of these comparisons is stated as a result.]

7.1 GETTING THE THRESHOLD RIGHT IS MOST OF THE PROBLEM

A learned head needs a firing threshold, and the obvious choices both fail. Thresholding the predicted λ at the label-space δ never fires at all: a regression head compresses its outputs toward the mean, so on TurnOnSinkFaucet the predicted λ peaks at .064 against a δ of .058 and only 0.12 percent of held-out predictions exceed it. Every switching run built this way executes fixed 16 with extra computation. Calibrating the threshold on held-out demonstrations instead fixes the scale but not the distribution, and produces online firing rates from 2 to 29 percent against a 4 percent target, because the policy visits states unlike the demonstrations. We therefore calibrate on deployment: a shadow run scores every replan without acting, and the threshold is set at the quantile of that online distribution which reproduces the oracle’s firing rate for the task. Each predictor then receives the same intervention budget as the oracle. Neither failure is visible in AUROC, which is unchanged across all three choices.

7.2 TASK SUCCESS

Table 8 is the main result of this section.

Three readings, and one caution.

The learned predictors improve on the vanilla policy on average, by 3 points for h1 and 8 points for h4, and they never collapse. The h4 mean of .62 closes most of the gap between the vanilla policy at .54 and the simulator oracle at .64, which is the strongest single number in favour of the approach here. The always $k=1$ column shows that this is not automatic: replanning every step for the whole episode averages .36, eighteen points below the vanilla policy, and reaches .04 on TurnOnSinkFaucet. The predictors enter that same single-step mode but only at selected moments and stay above the vanilla policy everywhere, so a rare and well placed switch is not the same intervention as a short fixed chunk.

The history predictor beats the single-clip predictor online on all four tasks, reversing their offline ranking in Table 4. The firing columns explain it. The two h1 cells that fire most, at .38 and .34, are the ones that gain least, and across the eight learned cells the correlation between firing rate and gain over the vanilla policy is -0.65 . The online advantage is calibration, not detection.

No controller reaches the best fixed chunk length on PickPlaceCounterToCabinet, where $k=8$ alone scores .78 and even the oracle reaches .77. On the three tasks whose best constant is already 16, the

Task	forked	rescued	broken	unchanged	p
PickPlaceCounterToCabinet	40	4	4	32	1.00
TurnOnSinkFaucet	22	0	0	22	1.00
combined	62	4	4	54	1.00

Table 9: Paired fork test. Rescued counts episodes that succeed only when the loop is closed at the fired moment, broken counts those that succeed only when it is ignored, and p is an exact two sided McNemar test. Success rates among forked episodes are identical in both branches, .625 and .455.

adaptive controllers gain a few points. Against a .10 run-to-run spread only TurnOnSinkFaucet with h4 is clearly outside noise.

The caution is that a success rate cannot separate the effect of an intervention from the difference between two independent sets of random episodes. The next experiment does separate them.

7.3 A PAIRED TEST OF THE INTERVENTION

We fork each episode at the decision point. An episode runs under the oracle until the first replan where λ crosses δ . At that point we snapshot the simulator and run the remainder twice from that identical state, once with the oracle acting and once with the oracle ignored so the policy continues with fixed 16. The branches share their entire history and differ only in the decision at the fired moment, which removes episode-to-episode variance completely.

The result is a clean null (Table 9). Closing the loop at a measured unstable moment rescues 6.5 percent of forked episodes and breaks an equal 6.5 percent. A paired comparison of time to success among episodes that succeed in both branches is also null and slightly favours ignoring the signal. The branches do diverge strongly, by up to a factor of five in steps to completion, so the null is not an artifact of the two arms staying identical.

This also explains the pattern in Table 8. The largest oracle number, .77 on PickPlaceCounterToCabinet, sits below a fixed chunk of 8 at .78, and a second seed of the faucet oracle gives .50 where the first gave .62. What looked like a causal gain in single unpaired cells does not survive pairing.

7.4 THE SAME CONCLUSION ON A SECOND POLICY CLASS

We ran the equivalent sweep with diffusion policies on LIBERO, MimicGen and robomimic: 42 predictor-driven cells on tasks that carry signal. Only 9 beat their fixed-16 baseline, and none beats the best constant chunk for its task. Where a shorter constant helps, using it everywhere is better than switching: nut assembly reaches .52 at $k=1$ against a best predictor cell of .46, and three-piece assembly reaches .78 at $k=1$ against .62. The damage is graded by how often the predictor fires, with a correlation of +0.58 between mean executed chunk length and change against baseline. Cells that fire most lose 23 points on average, cells that fire least are neutral. Those runs used demonstration-calibrated thresholds, so part of that damage is the calibration failure described above, but even the lightly firing cells average exactly zero benefit.

7.5 A COMPOSED TASK WITH A BUILT-IN REGIME CHANGE

Every task above is a single stage. If the hypothesis is right, switching should pay most where an episode is guaranteed to cross regimes. We build such an episode by forcing the cabinet shut at reset inside a PickPlaceCounterToCabinet environment, so one 1800-step episode contains an opening stage and then a pick-and-place stage, selected by the language instruction. Stage 1 ends on the environment’s own door-open predicate; full success needs the environment’s pick-and-place predicate as well.

Both learned predictors reach .48 against .44 for the best constant chunk, and the oracle is higher still at .54. The stage-1 column locates the effect: the regime-aware controllers get through the opening stage more often than any constant chunk, .78 and .79 against .68. This is the result the hypothesis predicts and the only place we observe it, which we read as support for the mechanism and against the generality of the intervention rather than the reverse.

Controller	full task	stage 1
fixed $k=16$	.38	.66
fixed $k=8$	.44	.68
fixed $k=4$	.22	.48
fixed $k=1$	.10	.62
h1 (16, 1)	.48	.66
h4 (16, 1)	.48	.78
simulator oracle (16, 1)	.54	.79

Table 10: Composed OpenCabinet then PickPlaceCounterToCabinet in one 1800-step episode, 50 episodes per cell. This is the only setting in the paper where switching beats the best constant chunk. The oracle cell is [pending: 28 of 50 episodes at the time of writing]. The learned cells select a stage-matched head using the environment’s stage predicate, which is privileged information the other cells do not receive; [pending: a pooled head over all four tasks, which needs no stage signal, is training and will replace these cells].

Task	controller	seed 0	seed 1	spread
PickPlace	vanilla	.58	.62	4
PickPlace	h1	.60	.70	10
PickPlace	oracle	.80 / .74	.60	20
TurnOnSinkFaucet	vanilla	.42	.54	12
TurnOnSinkFaucet	h1	.54	.66	12
TurnOnSinkFaucet	oracle	.62 / .50	.48	14

Table 11: Success across seeds for identical configurations, 50 episodes each, spread in percentage points. The vanilla baseline alone moves 12 points on TurnOnSinkFaucet and the PickPlace oracle moves 20, which is larger than any effect reported in this section. [pending: full 3-seed replication in progress, 8 of 32 cells]

Two caveats keep it from being a headline. The learned cells receive the stage boundary from the environment, and a pooled head trained over all four tasks, which needs no such signal, costs real accuracy: per-task AUROC falls to .69 to .80 from the .863 to .902 of single-task heads. Removing the privilege is not free, and the honest version of this table is still running.

7.6 SEED VARIANCE IS LARGER THAN THE EFFECTS

Every cell above is a single 50-episode run. Repeating four configurations at two further seeds gives the spread in Table 11, and it is sobering.

The .77 pooled oracle figure on PickPlaceCounterToCabinet does not reproduce at a third seed, and the vanilla baseline on TurnOnSinkFaucet moves further between seeds than the gap we attribute to switching. We therefore treat every single-seed number in this section as provisional, and none of the differences in Table 8 as established.

7.7 WHERE THIS LEAVES CLAIM 3

Regime-conditioned chunk length is, on current evidence, worth a few points over a fixed 16-step chunk and not worth anything over a properly swept constant. The intervention does not survive a paired causal test at the operating point we tested, which is a drop to single-step execution. Three things remain untested and are the natural next steps: milder switches such as $k=4$ at the fired moments, forking on tasks other than the two reported here, and the closed-loop axis, which is the axis that would tell us in advance whether closing the loop at an unstable moment can help at all.

8 CONCLUSION

Chunk length is usually treated as a per-task constant, and the quantity it trades against, the rate at which error compounds while the policy acts blind, is usually not measured at all. We measured it

directly as a finite-time maximal Lyapunov exponent per state, with a procedure that trains nothing and reads no policy internals.

On Claim 1 the evidence is strong. Across more than twenty datasets and three platforms, both stability regimes occur inside almost every episode, with several crossings each. The unstable segments are a small minority of timesteps, short, and concentrated at contact. The structure survives a fourfold change in probe size. Each task’s response to fixed chunk length follows its label profile, and this holds for two different policy classes.

On Claim 2 the evidence is also strong. Small heads reading only camera frames and proprioception recover the regime at 0.83 to 0.96 AUROC on confident stamps in domain, transfer across tasks at 0.72 to 0.77, and beat a privileged contact baseline by a wide margin. Temporal history does not help, which places the signal in the current configuration rather than in motion. Audited against our labels, the replan signals used by existing adaptive-horizon methods do not track the measured stability, and one of them is anti-correlated with it. The closed-loop axis is measured for four robomimic tasks and in progress on RoboCasa.

On Claim 3 the evidence is negative so far, and we prefer to report that plainly. Learned predictors driving the switch gain a few points over a fixed 16-step chunk, never collapse the way constant single-step execution does, but stay inside the run-to-run spread and never beat a properly swept constant chunk. A paired fork test, which removes episode variance by running both decisions from one simulator state, finds that closing the loop at a measured unstable moment rescues as many episodes as it breaks. A 42-cell sweep with diffusion policies on three other benchmarks agrees. Along the way we found two deployment failures that offline detection metrics hide: a regression head compresses its predictions so the label threshold never fires, and a threshold calibrated on demonstrations mis-fires by up to a factor of seven once the policy visits its own states.

The open question is the second stability axis. Switching should pay exactly where a segment is open-loop unstable and closed-loop stable, and we cannot yet say how common that combination is, because a closed-loop measurement needs both branches under policy control and a control for the policy’s own sampling noise. That measurement, milder switches than single-step execution, and paired tests on more tasks are the direct next steps.

REFERENCES

- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.
- Xiangdong Feng, Yuxuan Cheng, Chen Shi, Boyao Han, Yuxuan Yan, Yitong Hong, Zhuotao Tian, and Li Jiang. Denoising tells when to replan: Denoising-variance adaptive chunking for flow-based robot policies. *arXiv preprint arXiv:2606.03847*, 2026.
- George Haller. Lagrangian coherent structures. *Annual Review of Fluid Mechanics*, 47:137–162, 2015.
- Kartik Krishna, Steven L. Brunton, and Zhuoyuan Song. Finite time lyapunov exponent analysis of model predictive control and reinforcement learning. *IEEE Access*, 2023. arXiv:2304.03326.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Yuejiang Liu, Jubayer Ibn Hamid, Annie Xie, Yoonho Lee, Maximilian Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via closed-loop resampling. *arXiv preprint arXiv:2408.17355*, 2024.
- Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Ireteyio Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *Conference on Robot Learning (CoRL)*, 2023.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.

- Junnan Nie, Jiayi Li, Jiachen Zhang, Junyi Lao, Chenghao Liu, Tianle Zhang, and Songfang Huang. PACE: Phase-aware chunk execution for robot policies with action chunking. *arXiv preprint arXiv:2606.00537*, 2026.
- NVIDIA. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Stéphane Ross and J. Andrew Bagnell. Efficient reductions for imitation learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.
- Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- Max Simchowitz, Daniel Pfrommer, and Ali Jadbabaie. The pitfalls of imitation learning when actions are continuous. In *Conference on Learning Theory (COLT)*, volume 291 of *Proceedings of Machine Learning Research*, pp. 5248–5351, 2025. *arXiv:2503.09722*.
- Junhyuk So, Chiwoong Lee, Shinyoung Lee, Jungseul Ok, and Eunhyeok Park. Improving generative behavior cloning via self-guidance and adaptive chunking. *arXiv preprint arXiv:2510.12392*, 2025.
- Haoxuan Wang, Gengyu Zhang, Yan Yan, Ramana Rao Kompella, and Gaowen Liu. Vla knows its limits: Adaptive execution horizons for robot policies. *arXiv preprint arXiv:2602.21445*, 2026.
- Qingpeng Wen, Haomin Zhu, Yuepeng Zhang, Linzhong Xia, Bo Gao, and Zhuozhen Li. Adaptive action chunking for robotic imitation learning. *Biomimetics*, 11(5):316, 2026.
- Jiyao Zhang, Zimu Han, Junhan Wang, Xionghao Wu, Shihong Lin, Jinzhou Li, Hongwei Fan, Ruihai Wu, Dongjiang Li, and Hao Dong. HiPolicy: Hierarchical multi-frequency action chunking for policy learning. *arXiv preprint arXiv:2604.06067*, 2026.
- Thomas T. Zhang, Daniel Pfrommer, Chaoyi Pan, Nikolai Matni, and Max Simchowitz. Action chunking and exploratory data collection yield exponential improvements in behavior cloning for continuous control. *arXiv preprint arXiv:2507.09061*, 2025.
- Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023.
- Yuchi Zhao, Miroslav Bogdanovic, Arjun Sohal, Liyu Tao, Kourosh Darvish, Alán Aspuru-Guzik, Florian Shkurti, and Animesh Garg. Dynamic execution horizon prediction for chunk-based robot policies. *arXiv preprint arXiv:2606.11408*, 2026.

A ERROR GROWTH OVER THE EXECUTION HORIZON

This appendix expands the premise paragraph of Section 3: the statements we take from the literature, the two lines of algebra that specialize them to one executed chunk, and what our measurement of λ does and does not license.

What is quoted. Three results are used, none of them ours.

- (i) *Compounding from distribution shift.* Ross & Bagnell (2010), Theorem 2.1: if $\hat{\pi}$ has expected 0/1 loss at most ε under the expert’s own state distribution, then $J(\hat{\pi}) \leq J(\pi^*) + T^2\varepsilon$ over a horizon of T steps, and the quadratic is tight, since a system, cost and policy can be constructed that attain it. This is the distribution-shift form of compounding error and it assumes nothing about the dynamics.
- (ii) *The continuous-action obstruction.* Simchowitz et al. (2025) define f to be (C, ρ) -exponentially incrementally input-to-state stable when equation 1 holds for all pairs of initial states and input sequences, with $C \geq 1$ and $\rho \in [0, 1)$. Their Lemma 2.1 gives the benign direction: if the *closed loop* $(f, \hat{\pi})$ is (C, ρ) -stable and $\hat{\pi}$ is $L_{\hat{\pi}}$ -Lipschitz, the execution error is at most $\frac{C}{1-\rho}(2 + L_{\hat{\pi}})$ times the expert-distribution error, with no factor of the horizon at all. Their main theorem gives the

obstruction: there are imitation problems with smooth deterministic experts and stable dynamics on which any smooth, Markovian, simply-stochastic imitator suffers execution error $\exp(\Omega(T))$ times its training error. Closing the loop is not by itself enough; the closed loop has to be stable, and learning from expert data alone does not deliver that.

(iii) *Chunking as the remedy.* Zhang et al. (2025) analyze action chunking directly. Their Proposition 3.1 states that if the learned pair $(\hat{\pi}, \hat{f})$ is (C_{ISS}, ρ) -stable and the chunk length ℓ exceeds $\log(1/\rho)^{-1} \log(\text{poly}(L, C_{\text{ISS}}))$, then the chunked policy on the true dynamics is again exponentially stable, with constant $\log(1/\rho)^{-1} \text{poly}(L, C_{\text{ISS}})$ and rate $\rho^{1/2}$; their Theorem 1 then bounds trajectory error by a constant multiple of regression error, with the constant free of T . The mechanism is that a long enough open-loop chunk lets the plant’s own contraction wash out the mismatch between the learned and true dynamics before the next replan is committed. Note the direction: this is a lower bound on ℓ , and it presumes $\rho < 1$.

What is derived. The bounds in equation 2 and equation 3 follow from equation 1 by substitution and are not theorems from those works.

For the labeling probe, both branches start at the same state, so the first term of equation 1 vanishes, and only u_1 differs, so the sum has one nonzero term: $\|x_j - x'_j\| \leq C\rho^{j-1}\|u_1 - u'_1\| = C\sigma_u e^{(j-1)\lambda}$ with $\lambda := \log \rho$. Taking logarithms gives an affine function of j with slope λ , which is what Algorithm 1 fits.

For one executed chunk, the action error is present at every step because no replan intervenes, so $\|u_i - u'_i\| = \varepsilon$ for all $i \leq k$ and the sum is geometric, $C\varepsilon \sum_{i=0}^{k-1} e^{i\lambda} = C\varepsilon (e^{\lambda k} - 1)/(e^\lambda - 1)$, which is equation 2. Its three branches are: for $\lambda < 0$ the sum is increasing in k and converges, so $C\varepsilon/(1 - e^\lambda)$ bounds it for every k ; at $\lambda = 0$ the ratio has a removable singularity with value k , giving $C\varepsilon k$; and for $\lambda > 0$ the numerator dominates, leaving $C\varepsilon e^{\lambda k}/(e^\lambda - 1)$.

For the episode, write e_n for the deviation at the n th chunk boundary and model one replan as a contraction by γ , so that $e_{n+1} \leq \gamma(e^{\lambda k} e_n + C\varepsilon \frac{e^{\lambda k} - 1}{e^\lambda - 1})$. This is a linear recursion with gain $\gamma e^{\lambda k}$, bounded over $n = T/k$ chunks exactly when $\gamma e^{\lambda k} < 1$, that is when $k < \log(1/\gamma)/\lambda$. Combining it with the lower bound of (iii) gives the admissible window equation 3.

The threshold in amplification units. The automatic threshold δ of Section 3 is set from the λ distribution alone and never sees a chunk length, so where it lands in the units of equation 2 is a check rather than a construction. Table 12 converts it. On all four RoboCasa tasks the threshold sits where one 16-step chunk multiplies an action error by between 2.5 and 3.9, the top percentile of stamps sits at 4.5 to 6.9, and the worst stamps reach 19 to 107. Equivalently, holding the amplification at a stamp on the threshold to a factor of two would require chunks of 8 to 12 steps rather than 16.

Task	threshold		99th pct.		worst stamp	
	δ	$e^{16\delta}$	λ	$e^{16\lambda}$	λ	$e^{16\lambda}$
OpenCabinet	.085	3.9	.115	6.3	.222	35
OpenDrawer	.076	3.4	.121	6.9	.223	36
PickPlaceToCabinet	.065	2.8	.095	4.5	.184	19
TurnOnSinkFaucet	.058	2.5	.098	4.8	.292	107

Table 12: The measured λ distribution read as the per-chunk amplification factor $e^{\lambda k}$ of equation 2 at the executed chunk length $k=16$. The threshold column is the automatic δ of Table 1, which is computed from the below-median half of the λ distribution and has no knowledge of k .

What the correspondence does not license. Four caveats, in decreasing order of how much they matter.

λ is measured on an output, not on the state. The left-hand side of equation 1 is $\|x_j - x'_j\|$, the full state. Our label is the growth rate of a task metric $y = h(x)$, and composing a Lipschitz h with equation 1 bounds $\|y_j - y'_j\|$ by $L_h C \rho^{j-1} \sigma_u$, so the fitted slope estimates the rate along the observed coordinates rather than the state’s leading exponent. The two are not interchangeable here.

The labeler also records the slope on the full simulator state, and on RoboCasa the two are nearly uncorrelated: Spearman 0.21, 0.07, -0.02 and 0.01 on the four tasks, with the full-state slope having a *negative* median (-0.019 to -0.010) where the task-metric slope is near zero or positive. The full state settles while the coordinates the task cares about separate. We use the task metric deliberately, because whether the chunk still lands where it was aimed is a statement about those coordinates, but the label should be read as an output-space expansion rate and not as the plant’s leading Lyapunov exponent.

γ is assumed, not measured. Nothing in this paper measures the contraction a single replan buys. The closed-loop labels of Section 3 measure whether replanning helps at all at a state, which is the sign of the effect, not its magnitude, so equation 3 is a statement of shape rather than a formula we can evaluate. Measuring γ is the natural way to turn the switching rule from two levels into a schedule.

C and L are not measured either. The lower bound in equation 3 therefore cannot be evaluated for our systems, and we do not claim $k=16$ satisfies it. What the window says is that the two bounds move independently: the lower one is a property of the system and the learned model, the upper one of the state.

The rate is finite-time. λ is fitted over a $K=24$ step window and is a finite-time exponent in the sense of Haller (2015), not an asymptotic one. Figure 2(b) shows that neither divergence curve is a clean straight line; the slope summarizes the window, and equation 2 inherits that approximation.

B THE LABELING PROCEDURE IN FULL

Algorithm 1 is the procedure of Section 3 written out. It is the whole method: there is no training step and no policy query.

Algorithm 1 Open-loop stability label at one state

Require: simulator state s_{t_0} , recorded actions $a_{t_0}, \dots, a_{t_0+K-1}$, horizon K , branches N , scale σ_u

- 1: restore the simulator to s_{t_0}
- 2: **for** $j = 0$ to $K - 1$ **do** ▷ nominal branch
- 3: step the simulator with a_{t_0+j} ; record $y_j \leftarrow$ task metric
- 4: **end for**
- 5: **for** $n = 1$ to N **do**
- 6: $\tilde{a} \leftarrow a_{t_0}$; $\tilde{a}[\text{translation}] += \sigma_u \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I_3)$
- 7: restore the simulator to s_{t_0}
- 8: **for** $j = 0$ to $K - 1$ **do** ▷ perturbed branch: same actions after the first
- 9: step with $\tilde{a}$ if $j = 0$ else a_{t_0+j} ; record $y_j^{(n)}$
- 10: $d_j^{(n)} \leftarrow \|y_j^{(n)} - y_j\|$
- 11: **end for**
- 12: **end for**
- 13: **return** $\lambda \leftarrow$ OLS slope of $\frac{1}{N} \sum_n \log \max(d_j^{(n)}, 10^{-12})$ against j

Settings. Every dataset in this paper uses $K=24$, $N=8$, a stride of 2 between labeled stamps, and perturbs the three translation dimensions only. The gripper dimension is never perturbed, because a perturbation there changes which discrete contact mode the branch enters and measures a different thing.

Task metric. The metric y is the quantity whose divergence defines the label, and it differs by platform because the observation spaces do. On robomimic, MimicGen and LIBERO it is the end-effector position concatenated with the task object’s observation vector. On RoboCasa it is the end-effector position concatenated with the gripper joint positions, because the kitchen environments have no single distinguished object observation. Both are norms over quantities in metres, so λ carries units of inverse control steps in both cases. The change of metric between platforms means δ , and therefore the unstable fraction, is not directly comparable across the two groups; the within-episode structure that Claim 1 rests on is.

Action ordering. The RoboCasa demonstrations are stored in a different action order from the one the controller consumes:

$$\text{stored } [\text{base}_3, \text{torso}_1, \text{mode}_1, \text{arm}_6, \text{grip}_1] \rightarrow \text{consumed } [\text{arm}_6, \text{grip}_1, \text{base}_3, \text{torso}_1, \text{mode}_1].$$

An earlier version of our labels replayed the stored order directly, which drives the arm with base commands and makes the nominal branch itself a failure. Replay validation caught it: the remapped order reproduces the demonstration end state to within 4 to 7 centimetres, the raw order misses by about a metre. Every RoboCasa number in this paper comes from labels regenerated after the fix. We report this because a silent action-order mismatch produces labels that look plausible, with a sensible λ distribution and a sensible threshold, and is invisible without an explicit replay check.

Determinism. The label is reproducible to the last digit only if the simulator call sequence is reproduced exactly, including calls that read nothing. Re-running the procedure while omitting a redundant `reset_to` shifts MuJoCo’s warm-start cache, which perturbs the constraint solver and moves λ by a few thousandths, enough to flip stamps near δ . Regenerating the divergence curves of Figure 2 with the call sequence reproduced exactly recovers the stored labels with a maximum absolute difference of 0 over the 25 stamps checked.

C THE PERTURBATION SCALE σ_u

How it is set. The intent stated in Section 3 is that σ_u should be the trained policy’s validation action RMSE, so that the injected perturbation is the size of the error the policy actually makes. Table 13 gives the value used for every dataset. For the diffusion-policy platforms this rule is applied directly, using the position-dimension RMSE of the trained checkpoint on held-out demonstrations. For RoboCasa it is not: all four tasks use $\sigma_u=0.05$, chosen before the GR00T checkpoint was evaluated and kept for consistency across the sweep.

Platform / dataset	σ_u	source
RoboCasa, all four tasks	0.05	fixed, see below
robomimic can	0.225	policy RMSE
robomimic lift	0.165	policy RMSE
robomimic square	0.220	policy RMSE
robomimic tool_hang	0.164	policy RMSE
MimicGen coffee preparation	0.114	policy RMSE
MimicGen nut assembly	0.103	policy RMSE
MimicGen square	0.220	policy RMSE, square ph
MimicGen stack	0.203	robomimic mean
LIBERO-10, all ten tasks	0.203	robomimic mean

Table 13: Perturbation scale per dataset. Policy RMSE is the position-dimension action RMSE of the trained diffusion policy on held-out demonstrations. Two datasets use the mean over the robomimic policies because their own policy was not yet trained when the labels were produced.

The RoboCasa gap. Applying the stated rule to RoboCasa would give a much larger perturbation than the one we use. The GR00T N1.5 checkpoint’s position-dimension action RMSE on held-out demonstrations is 0.565 on OpenCabinet, 0.426 on OpenDrawer, 0.479 on PickPlaceCounterToCabinet and 0.299 on TurnOnSinkFaucet, pooling to 0.467, an order of magnitude above the 0.05 used for labeling. The sensitivity analysis below covers a fourfold range around 0.05 and does not reach the rule-implied value, so it does not by itself justify the choice at that scale. [pending: relabel at least one RoboCasa task at the measured RMSE and report whether the regime structure survives; if it does not, the honest statement is that σ_u is a free parameter of the label rather than a policy-derived one]

Sensitivity. Table 14 relabels all four RoboCasa tasks at $\sigma_u \in \{0.025, 0.05, 0.10\}$. The threshold δ moves with the probe, as it must, since a larger kick produces larger separations everywhere. What matters is that the per-stamp ordering of λ is nearly preserved and the resulting unstable flag barely

moves: adjacent settings agree on 96.6 to 99.4 percent of stamps. The structure the paper relies on is therefore a property of the dynamics, not of the probe size, over this range.

Task	δ at σ_u			Spearman ρ of λ			agree
	.025	.05	.10	.025/.05	.05/.10	.025/.10	%
OpenCabinet	.100	.085	.067	.98	.96	.90	99.1–99.4
OpenDrawer	.088	.076	.061	.96	.93	.85	99.0–99.3
PickPlaceToCabinet	.078	.065	.046	.97	.95	.89	96.6–98.4
TurnOnSinkFaucet	.068	.058	.046	.95	.93	.82	97.2–98.3

Table 14: Relabeling all four RoboCasa tasks across a fourfold range of σ_u , on the same 20,198 to 34,133 stamps per task. The agree column is the fraction of stamps on which the unstable flag matches the $\sigma_u=0.05$ labeling, given as the range over the two off-centre settings.

D THRESHOLD SELECTION

Section 7 states that setting the firing threshold for a learned head is most of the problem. Table 15 gives the numbers behind that claim.

Task	δ	label-space δ		online	
		$\max \hat{\lambda}$	% over	target	thresh
OpenCabinet	.085	.097	0.02%	.06	.045
OpenDrawer	.076	.058	0.00%	.14	.021
PickPlaceToCabinet	.065	.076	0.02%	.11	.030
TurnOnSinkFaucet	.058	.064	0.12%	.12	.037

Table 15: Why the obvious threshold does not work, h1 head. Applying the label-space δ to the head’s predicted $\hat{\lambda}$ fires on 0.12 percent of held-out stamps at best and on none at all for OpenDrawer, because the regression head compresses its outputs toward the mean: the maximum $\hat{\lambda}$ it ever emits barely reaches δ . The online columns give the deployment calibration we adopt, where the threshold is the quantile of a shadow run’s score distribution that reproduces the oracle’s firing rate for that task. Target and achieved rates agree to within 0.1 percentage point on all eight predictor cells.

Three points are worth separating. First, the failure is not a detection failure: AUROC is identical under all three threshold choices, because AUROC is invariant to any monotone rescaling of the score. An offline metric cannot see this problem at all. Second, calibrating on held-out demonstrations fixes the scale but not the distribution, and gives online firing rates from 2 to 29 percent against a 4 percent target, because the policy visits states unlike the demonstrations. Third, the deployment calibration we adopt requires a shadow run per task and predictor, which is a real cost and one the oracle does not pay.

E PREDICTOR ARCHITECTURE AND TRAINING

Features. Frozen V-JEPA 2 ViT-L (vjepa2-vitl-fpc64-256), never fine-tuned. For a stamp at time t we take the 16 camera frames ending at t , padded at episode start, resize to 256, and keep a pooled token grid of 8 temporal positions $\times$ 4 $\times$ 4 spatial positions $\times$ 1024 channels, that is 128 tokens per stamp, stored in fp16. Proprioception is end-effector position, quaternion and gripper joint positions over the same 16-step window.

Head. A learned query attends over the token grid with 8-head multi-head attention, followed by a LayerNorm. Proprioception passes through a single-layer GRU with 64 hidden units. The pooled visual vector and the final GRU state are concatenated and passed through a two-layer MLP, hidden width 256, GELU, dropout 0.1, to two scalar outputs: a regression estimate of λ and a classification logit for $\lambda > \delta$. The h4 variant is the same network reading four clips spaced 16 frames apart with a learned per-clip position embedding added before attention; h1 is the same code path with one clip.

Training. AdamW, learning rate 10^{-3} , weight decay 10^{-4} , cosine annealing over 40 epochs, batch size 128. The loss is a Huber loss with $\delta_H=0.1$ on λ over all stamps plus a positive-weighted binary cross-entropy on confident stamps only, that is those with $|\lambda| > \delta$, since the deadband label is ambiguous by construction. The split is by demonstration, not by stamp, with 20 percent of demonstrations held out, so no frame from a validation episode is seen in training. Three seeds per configuration; the seed controls initialization, shuffling and the train/validation split together.

Task	AUROC, confident		Spearman with λ	
	h1	h4	h1	h4
OpenCabinet	.885 $\pm$.021	.858 $\pm$.014	.43	.34
OpenDrawer	.863 $\pm$.023	.838 $\pm$.010	.24	.16
PickPlaceToCabinet	.895 $\pm$.016	.887 $\pm$.006	.40	.31
TurnOnSinkFaucet	.902 $\pm$.022	.895 $\pm$.012	.33	.28

Table 16: Predictor quality on held-out demonstrations, mean and standard deviation over three seeds. These are the four RoboCasa rows of Table 4 with the regression quality added, which is the per-stamp Spearman correlation between predicted and measured λ rather than a ranking of the unstable stratum. The single-clip head is ahead of the four-clip head on every task and on both metrics, though the AUROC gap exceeds the seed spread on only two of the four tasks, OpenCabinet and OpenDrawer; the Spearman column is the clearer separation. Training sets are 16,090 to 27,974 stamps and validation sets 4,108 to 6,159 stamps.

F COMPUTE

Labeling is the dominant cost and is CPU-only: no rendering, no network, no gradient. One RoboCasa task, 200 demonstrations and 20,198 to 34,133 labeled stamps, takes 2.8 to 6.5 hours on 24 worker processes. Each stamp costs $(N+1) \times K = 216$ simulator steps plus two state restores, so the cost scales linearly in stamps, in N and in K , and the stride is the only knob that changes stamp count. Feature extraction is one forward pass of a frozen ViT-L per stamp on one GPU. Head training is minutes on one GPU, the heads being small relative to the frozen encoder. The simulator oracle of Claim 2 has the same per-decision cost as one label, 216 steps, which is why it is a bound rather than a deployable controller.